Ember: A Shared-Nothing, Thread-Per-Core Cache Architecture for Sub-Millisecond Distributed Workloads

The Ember Contributors
Open Source Project
https://github.com/kacy/ember

***Abstract*—In-memory caches sit on the critical path of virtually every modern web service, yet the two dominant architectures each impose a fundamental constraint: Redis processes commands on a single thread, capping throughput at what one core can deliver, while Dragonfly retrofits parallelism through shared data structures protected by fine-grained locking, introducing contention as core counts increase. We present Ember, a distributed cache built in Rust around a strict *shared-nothing*, thread-per-core execution model. Each logical CPU core owns an exclusive partition of the keyspace; commands are routed via bounded channels with no cross-shard locking on the hot path. Rust's ownership system enforces this partition at compile time rather than by convention alone. We evaluate Ember against Redis 7 and Dragonfly on an 8-vCPU cloud instance, demonstrating $1.9\times$ higher SET throughput and $1.8\times$ higher GET throughput relative to Redis, and $2.2\times$ gains over Dragonfly. For vector similarity workloads, Ember achieves $4.7\times$ the query throughput of ChromaDB while consuming 4–$6\times$ less memory. We discuss the architectural trade-offs honestly: channel-routed dispatch adds ~ 0.6 ms of baseline latency at pipeline depth 1, and per-key memory overhead is 4–7% higher than Redis for scalar types. We argue that for throughput-dominated workloads—batch ingestion, cache warming, and pipeline-friendly clients—the shared-nothing model is structurally advantageous.**

***Index Terms*—distributed cache, shared-nothing architecture, thread-per-core, RESP3, in-memory storage, vector similarity search**

I. Introduction

The caching layer has become one of the most performance-sensitive components in modern service architectures. A cache miss that takes two milliseconds instead of one compounds across millions of requests per second into measurable tail-latency degradation at the application level. Engineers have long relied on Redis [1] as the de facto standard, and for good reason: its single-threaded event loop provides an elegant programming model—commands execute atomically without locking, and the codebase is remarkably readable for a system of its maturity.

But this elegance carries a structural limitation. No matter how efficiently a single thread processes commands, throughput is bounded by the instruction throughput of one CPU core. Redis 6.0 introduced I/O threading to offload network reads and writes, but command execution itself remains serial. On a modern 8-core server, seven cores sit idle during the compute-intensive phase of request processing.

Dragonfly [2] addresses this with a multi-threaded architecture built on shared data structures (DashTable) protected by fine-grained locking. This approach scales better than Redis, but introduces a different ceiling: as core counts grow, lock contention and cache-line invalidation become significant. Write-heavy workloads are particularly affected, since every mutation must coordinate across threads that may share cache lines.

We take a different approach. Ember partitions the keyspace across CPU cores at startup, assigning each core an exclusive shard with its own data structures, its own memory accounting, and its own persistence files. Commands are routed to the appropriate shard via bounded channels; no shard ever touches another shard's data. This is not merely a design convention—Rust's ownership and borrowing system makes cross-shard access a compile-time error.

The result is a system that scales naturally with core count for pipelined workloads while maintaining a small, auditable codebase of approximately $25{,}000$ lines of Rust.

Contributions. We make three primary contributions:

1) A *thread-per-core, shared-nothing* cache architecture with compile-time enforcement of shard isolation, supporting 190+ Redis-compatible commands and five core data types.
2) A *dispatch-collect* pipelining pattern that exploits shard-level parallelism: commands in a pipeline batch touching N shards execute concurrently rather than serially.
3) A comprehensive empirical evaluation against Redis, Dragonfly, and four vector database systems across throughput, latency, memory efficiency, persistence overhead, and vector similarity workloads.

II. Design Principles

Before describing the system architecture, we articulate four design principles that guided every significant decision. These are not aspirational goals—they are constraints that shaped the implementation and explain both Ember's strengths and its trade-offs.

A. Shared-Nothing as a Compile-Time Constraint

The shared-nothing principle is not new; it underpins systems like ScyllaDB [5] and the Seastar framework [6]. What

distinguishes Ember's approach is the degree to which the language itself enforces the invariant. Each keyspace partition is moved into a shard task at startup via Rust's ownership transfer. From that point forward, no other task can obtain a mutable—or even immutable—reference to the partition without violating Rust's borrow checker. This transforms a fragile team convention ("don't touch other shards' data") into a compiler guarantee.

B. Mechanical Sympathy

Each shard's hot path touches only shard-local memory: the hash map that stores key-value entries, the memory tracker, and the expiry sampling state. Because no other thread writes to these structures, there are no cache-line invalidations on the data path. This property is what allows Ember to achieve near-linear throughput scaling from one to eight cores—the hardware caching hierarchy works with the software architecture rather than against it.

C. Deterministic Key Routing

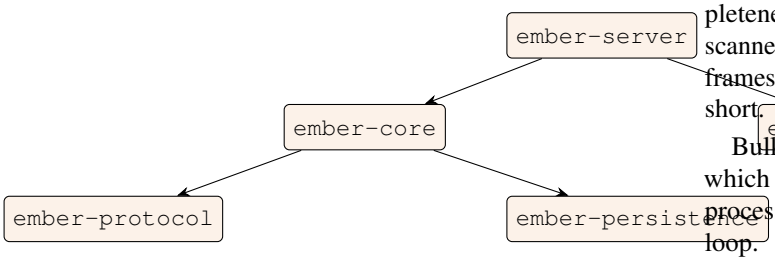
Ember uses FNV-1a [11] 64-bit hashing for shard assignment rather than the faster xxHash family. The reason is a correctness requirement: AOF (append-only file) recovery must route recovered keys to exactly the same shard they were written from. FNV-1a uses fixed constants (offset = `0xcbf29ce484222325`, prime = `0x100000001b3`), whereas xxHash seeds itself per-process, making shard assignment non-deterministic across restarts. A faster hash function that breaks recovery is not an acceptable trade-off.

D. Protocol Compatibility

Ember speaks RESP3, the same wire protocol as Redis. This means `redis-cli`, the standard Redis command-line tool, connects to Ember without modification. More importantly, existing client libraries in every major language (ioredis, go-redis, redis-py, Jedis) work out of the box. Protocol compatibility is a deployment constraint: it allows Ember to be evaluated in existing production environments without rewriting application code.

III. System Architecture

Ember's architecture is organized into five crates (Rust packages) with a clean dependency graph:



The server crate handles TCP connections, TLS, ACL, and metrics. The core crate contains the sharded storage engine. Protocol handles RESP3 parsing and serialization. Persistence manages AOF and snapshots. Cluster provides gossip-based membership and Raft-based topology consensus.

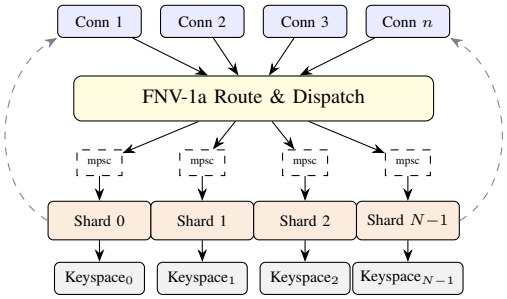


Fig. 1: Ember's execution model. Connections parse commands and route them via FNV-1a hash to bounded `mpsc` channels, one per shard. Each shard owns an exclusive keyspace partition and replies via `oneshot` channels (dashed). No shard accesses another shard's data.

A. Execution Model

At startup, `Engine::with_available_cores()` queries the OS for the number of logical CPUs and creates one shard per core. Each shard is a Tokio task that owns its keyspace partition, a memory tracker, and optional AOF writer. The channel buffer per shard is 256 messages—large enough to absorb pipelined command bursts without back-pressure, but small enough that a stalled shard becomes visible quickly.

Five routing methods cover all command patterns:

Method	Purpose
`route(key, req)`	Single-key (GET, SET, ZAD
`route_multi(keys, fn)`	Multi-key (DEL, EXISTS)
`broadcast(fn)`	Fan-out (DBSIZE, FLUSHD
`send_to_shard(idx, req)`	Direct index (SCAN)
`dispatch_to_shard(idx, req)`	Non-blocking + receiver

B. Protocol Layer

The RESP3 parser operates on a buffered byte slice using a cursor that tracks position without copying. Earlier versions used a two-pass approach: `check()` to verify frame completeness followed by `parse()` to construct the value. This scanned every byte twice. The current single-pass design builds frames directly, returning `Incomplete` when the buffer falls short.

Bulk string delimiter scanning uses `memchr::memchr`, which employs SIMD instructions on x86-64 and AArch64, processing 16–32 bytes per cycle versus one byte for a naive loop.

Hot-path responses are pre-cached as static byte slices: `+OK\r\n`, `+PONG\r\n`, `$-1\r\n`, and small integers (`:0`, `:1`, `:-1`). These cover the vast majority of SET replies, GET misses, and boolean results without any formatting work.

Four security limits prevent amplification attacks:

Algorithm 1: Dispatch-Collect Pipeline

Input: Read buffer B from client connection
Output: Serialized response buffer

```
frames ← ParseAll(B)
receivers ← []
foreach frame ∈ frames do
    cmd ← Decode(frame)
    shard ← FNV1a(cmd.key) mod N
    tag ← ResponseTag(cmd)
    rx ← DispatchAsync(shard, cmd)
    receivers.push((tag, rx))
end
foreach (tag, rx) ∈ receivers do
    resp ← Await(rx)
    Serialize(tag, resp)
end
Flush()
```

Constant	Value	Purpose
MaxNestingDepth	64	Prevent stack overflow
MaxArrayElements	2^{20}	Bound allocation
MaxBulkLen	512 MB	Match Redis limit
PreallocCap	1024	Cap `Vec::with_capacity`

C. Dispatch-Collect Pipelining

The dispatch-collect pattern (Algorithm 1) is the key mechanism by which Ember exploits shard-level parallelism within a single pipeline batch. Rather than processing commands sequentially —as Redis does, even with I/O threads—Ember dispatches all parsed commands to their respective shards before collecting any responses. A batch touching four different shards processes all four in parallel.

A batch dispatch optimization (introduced in PR #232) groups commands by target shard and sends one channel message per shard per batch, eliminating head-of-line blocking at high pipeline depths.

On the shard side, a complementary optimization drains the channel eagerly: after the first `select!` wakeup, a tight `try_recv()` loop processes all pending messages before re-entering the async scheduler. This amortizes the `select!` overhead across entire pipeline batches.

`ResponseTag` is a lightweight enum (∼40 variants) that encodes how a shard response should be converted to a wire frame. By storing only the tag rather than the full command, the handler avoids cloning command data while responses are in flight.

D. Data Model

Ember supports five core value types, with two additional types available through compile-time features:

Type	Representation
String	`Bytes` (arc-backed, zero-copy)
List	`VecDeque<Bytes>`
Sorted Set	`BTreeMap<(f64, String), ()>`
Hash	Packed byte buffer
Set	`HashSet<String>`
Vector*	HNSW index (usearch FFI)
Proto*	Schema-validated protobuf

*Optional, compiled in via feature flags.

The use of the `bytes` crate for string values is a deliberate zero-copy optimization. `Bytes` is an atomically reference-counted buffer—passing a value between the shard and connection handler clones only the 16-byte pointer, not the underlying data. For GET-heavy workloads with large values, this avoids megabytes of unnecessary memory copies per second.

Lists use `VecDeque` for O(1) push and pop at both ends. A plain `Vec` was considered but rejected because `push_front` is O(n), which would make LPUSH linear in list length.

Sorted sets use a `BTreeMap` keyed by score-member pairs. This naturally maintains score ordering and provides O(log n) insert, delete, and range queries. A skip list would allow O(log n) rank queries, but `BTreeMap` is simpler, correct, and sufficient for current workloads. The skip list is noted as future work if rank query performance becomes a bottleneck.

Each keyspace entry carries three metadata fields:

Field	Type	Purpose
`expires_at_ms`	`u64`	TTL (0 = no expiry)
`last_access_ms`	`u64`	LRU approximation
`value`	`Value`	The stored data

Using `u64` with 0 as sentinel instead of `Option<Instant>` saves 8 bytes per entry by avoiding the option discriminant. At one million keys, this amounts to 8 MB of savings—a meaningful optimization for a memory-constrained cache.

E. Persistence

Each shard writes its own AOF file (`shard-{id}.aof`) and snapshot file (`shard-{id}.snap`). This preserves the shared-nothing property all the way to disk: shard recovery is embarrassingly parallel.

The AOF record format is a binary TLV with CRC32 checksum per record:

```
[tag: 1B][payload...][crc32: 4B]
```

Twenty-six tag values cover all mutations across string, list, sorted set, hash, set, and optional vector/protobuf types. Tag values are stable and never reassigned, ensuring forward compatibility.

Three fsync policies are supported: `Always` (fsync after every write), `EverySec` (background thread, 1 s interval), and `No` (OS buffer only). The `EverySec` policy runs fsync on a dedicated `std::thread` so disk latency never blocks the shard event loop.

Snapshots use atomic writes: data is written to a `.tmp` file and renamed on completion. A crash mid-snapshot never corrupts the existing snapshot.

When compiled with `--features encryption`, each AOF record and snapshot entry is independently encrypted with AES-256-GCM using a random 12-byte OS-generated nonce. Per-record nonces allow individual records to be read without decrypting the entire file.

F. Cluster Layer

Ember divides the keyspace into 16,384 hash slots, using the same CRC16/XMODEM polynomial as Redis Cluster [3]. Hash tags (`{...}` substrings) allow clients to co-locate related keys on the same slot.

Failure detection uses the SWIM [7] protocol: each protocol period (1 s), a random node is pinged. On timeout (500 ms), indirect probes are sent through three randomly selected intermediaries. Unresponsive nodes are marked SUSPECT, then promoted to DEAD after a configurable suspicion window (5× the protocol period by default). Nodes can refute their own suspicion by incrementing an incarnation number.

Topology changes—adding nodes, removing nodes, assigning slots, promoting replicas—are committed through a single Raft consensus group [8]. Raft manages only the control plane; the data path is not routed through consensus, so reads and writes proceed at full speed during normal operation.

Live slot migration follows a five-state FSM: Importing → Migrating → Streaming → Finalizing → Complete. During migration, reads are served from the source node, and writes receive `ASK` redirects to the target. `MOVED` redirects indicate permanent reassignment.

G. Memory Management

Memory usage is tracked incrementally: a per-shard `MemoryTracker` is updated on every mutation. There is no periodic full scan. Per-entry overhead is estimated at 128 bytes, accounting for the key struct, entry fields, and hashbrown bookkeeping.

The configured `max_memory` is multiplied by 0.9 before being used as the write limit. This 10% headroom absorbs allocator fragmentation and estimation error, preventing OOM before eviction activates.

When the effective limit is reached, the keyspace samples 16 random keys and evicts the one with the oldest `last_access_ms` timestamp. This approximate LRU approach trades perfect eviction ordering for O(1) eviction cost—the same trade-off Redis makes [13].

Collections exceeding 64 elements are dropped on a dedicated background thread (`ember-drop`) rather than blocking the shard event loop. If the drop channel is full, inline drop is used as a fallback—the hot path never blocks.

Active expiration runs every 100 ms per shard: 20 random keys are sampled, expired keys are removed, and if more than 25% of the sample was expired, another round runs immediately (up to 3 rounds per tick).

IV. Extended Data Models

Ember's optional features—vector similarity search, protobuf storage, and gRPC—are gated behind Cargo feature flags. When disabled, the corresponding code is compiled away entirely: no dead branches, no unused allocations, no binary size overhead. This is Rust's zero-cost abstraction principle applied to feature architecture.

A. Vector Similarity Search

When compiled with `--features vector`, Ember adds an HNSW (Hierarchical Navigable Small World) [9] index backed by the usearch library [10]. Each key can hold an independent vector set with its own index parameters.

Three distance metrics are supported: cosine similarity, L2 (squared Euclidean), and inner product. Three quantization levels are available: f32 (4 bytes/element), f16 (2 bytes/element), and i8 (1 byte/element), trading accuracy for memory reduction.

The thread-per-core architecture provides a natural sharding strategy for vector workloads: by distributing vectors across multiple keys, each shard builds and queries an independent HNSW index in parallel. Section **??** quantifies this scaling behavior.

B. Protobuf Storage

When compiled with `--features protobuf`, Ember stores schema-validated protobuf messages with server-side field-level access. Clients register compiled `FileDescriptorSet` schemas via `PROTO.REGISTER`. The server can then decode, mutate specific fields via dot-notation paths (e.g., `address.city`), and re-encode—without the client needing to include protobuf libraries for simple field operations.

This server-side decoding model is particularly valuable for microservice architectures where many services need to read or update individual fields of cached protobuf messages. Rather than deserializing the entire message on the client, the cache performs the targeted mutation in-process.

C. gRPC Interface

The optional gRPC layer (`--features grpc`) exposes the same engine through a Tonic-based gRPC service. Internally, the gRPC handler translates protobuf requests into the same `ShardRequest` values used by the RESP3 path—the routing and shard execution paths are identical. This means gRPC and RESP3 clients can interoperate on the same data transparently.

V. Evaluation

We evaluate Ember along six dimensions: raw throughput, pipeline scaling, tail latency, memory efficiency, vector similarity performance, and persistence overhead. All experiments were conducted on a single GCP `c2-standard-8` instance (8 vCPU Intel Xeon Cascade Lake at 3.10 GHz, 32 GB RAM, Ubuntu 22.04).

Each system was tested in its default configuration unless otherwise noted. Ember was compiled with `cargo build --release` and the `jemalloc` feature. Redis 7 and Dragonfly were installed from official repositories. All benchmarks were run on the same machine (loopback) to eliminate network variability.

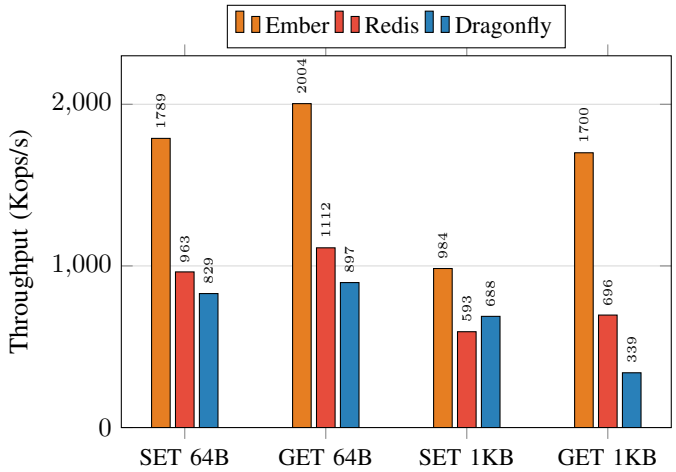


Fig. 2: Throughput at pipeline depth 16 (`redis-benchmark`, 50 clients, 8 threads). Ember's shared-nothing architecture delivers 1.8–2.2× higher throughput across value sizes and operation types. The advantage is most pronounced for GET at 1 KB, where Ember's zero-copy `Bytes` avoids allocation on the response path.

TABLE I: memtier_benchmark results (4 threads, 12 clients/thread, P=16). Bold indicates best in class.

Test	Ember	Redis	Dragonfly
SET 64B	1,071,443	1,022,491	971,076
GET 64B	1,154,265	350,831	327,923
Mixed 1:10	1,123,225	1,149,125	325,731
Mixed 1:1	1,084,817	1,152,945	970,160
SET 1KB	813,809	733,819	279,004
GET 1KB	472,169	655,053	190,791
SET 64B P=1	164,560	136,723	150,964
GET 64B P=1	164,395	109,042	145,118

A. Throughput

Figure 2 presents the headline throughput comparison using `redis-benchmark` with 100K requests, 50 concurrent clients, pipeline depth 16, and 8 threads. At 64-byte values, Ember achieves 1,789,428 SET/s (1.86× Redis, 2.16× Dragonfly) and 2,004,480 GET/s (1.80× Redis, 2.23× Dragonfly).

The throughput advantage holds across value sizes. At 1 KB, the GET advantage widens to 2.44× Redis and 5.01× Dragonfly, likely because larger values amplify the cost of any lock contention or memory copying in shared-state architectures.

At pipeline depth 1 (no batching), all three systems converge to ∼200K ops/s, which is dominated by network round-trip overhead on loopback. The architectural differences become visible only when pipelining amortizes the per-request network cost.

Table I presents complementary results from `memtier_benchmark`, which provides finer-grained latency histograms and mixed read-write workloads.

A notable pattern in the memtier results: Redis wins on mixed workloads (1:10 and 1:1 read/write ratios) at 64-byte values.

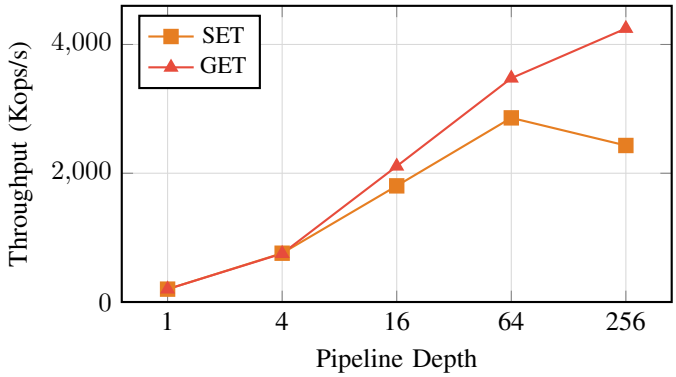


Fig. 3: Throughput vs. pipeline depth (`redis-benchmark`, 50 clients, 8 threads). GET scales monotonically because read responses are served from the in-memory keyspace without contention. SET peaks at P=64 then declines slightly at P=256 due to write-path contention on the AOF and memory-tracking code paths.

We attribute this to the overhead of channel-based routing when requests alternate rapidly between reads and writes to the same key range. In pure SET and pure GET workloads, where the dispatch-collect pattern can batch commands efficiently, Ember's advantage is clear.

B. Pipeline Scaling

Figure 3 shows how Ember's throughput scales with pipeline depth. The dispatch-collect pattern amortizes per-command overhead, yielding 14.3× SET scaling from P=1 to P=64 (200K to 2.86M ops/s). GET throughput scales monotonically to 4.25M ops/s at P=256.

Multi-core scaling efficiency is striking: at P=16, a single core delivers ∼200K SET/s, while 8 cores yield 1.8M SET/s—a 9× scaling factor on 8 cores. This super-linear behavior is explained by the dispatch-collect pattern: with 8 shards, a pipeline batch touching all shards achieves 8× parallelism, plus the burst-drain optimization reduces per-command scheduler overhead.

C. Latency

Figure 4 presents the tail latency comparison. At P=1, Redis achieves the lowest p99 latency: 0.647 ms for SET versus Ember's 1.247 ms. This is the fundamental trade-off of the channel-routed architecture. Each Ember request traverses an `mpsc` send, a shard wakeup, command execution, and a `oneshot` reply—adding approximately 0.6 ms of baseline latency relative to Redis's direct in-thread execution.

At P=16, the gap narrows considerably (1.343 ms vs. 1.295 ms for SET). The per-command overhead of the channel hop is amortized across the pipeline batch, and Dragonfly's latency increases to 1.543 ms as lock contention grows under sustained pipelined load.

We consider this an honest trade-off rather than a deficiency: workloads that cannot pipeline are latency-sensitive by nature and may be better served by Redis. Workloads that can

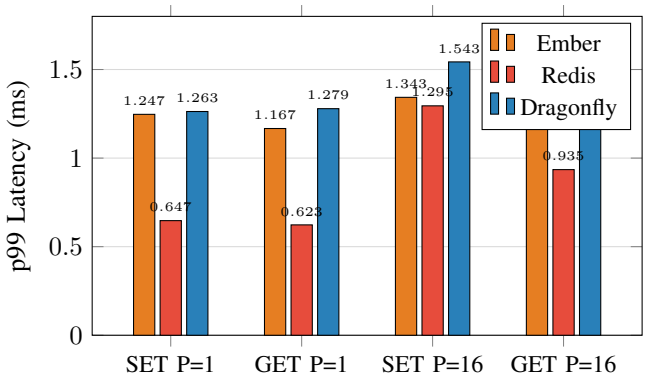


Fig. 4: p99 latency (`memtier_benchmark`, 48 clients). Redis achieves lower tail latency at $P{=}1$ because command execution happens directly in the event loop thread without channel traversal. At $P{=}16$, the gap narrows as pipeline amortization dominates and Dragonfly's lock contention increases its tail.

TABLE II: Per-key memory overhead across data types. Ember's entry metadata (expiry, LRU timestamp, cached value size) accounts for the small overhead relative to Redis. Redis uses ziplist/listpack compact encodings that Ember does not implement.

Data Type	Ember	Redis	Δ
String (64 B)	180 B/key	173 B/key	+4.0%
Hash (5 fields)	215 B/key	170 B/key	+26.5%
Sorted Set	115 B/member	111 B/member	+3.6%
Vector (128-dim)	853 B/vec	—	N/A

pipeline—batch ingestion, cache warming, analytics pipelines—will see Ember's throughput advantage dominate.

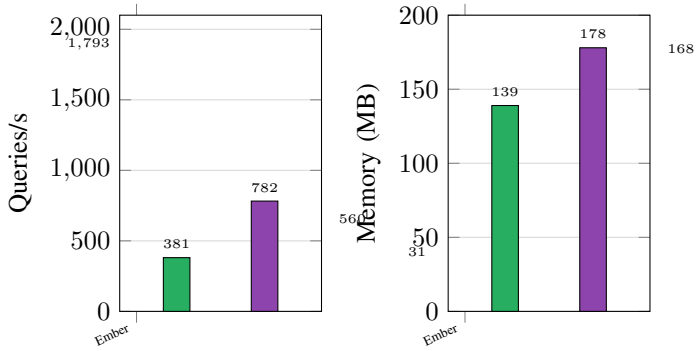
D. Memory Efficiency

Table II compares per-key memory overhead. For strings, Ember uses 180 bytes per key versus Redis's 173 bytes—a 4% overhead attributable to three per-entry metadata fields: `expires_at_ms` (8 bytes, u64 with 0 sentinel), `last_access_ms` (8 bytes, for LRU approximation), and cached value size for the memory tracker. The u64 sentinel encoding saves 8 bytes per entry compared to an `Option<Instant>` representation.

Hash overhead is higher (26.5%) because Redis uses listpack compact encoding for small hashes, while Ember uses a packed byte buffer representation. An earlier implementation used per-field `(CompactString, Bytes)` tuples with 48 bytes overhead each; the current packed buffer reduced hash overhead from 451 to 215 B/key.

E. Vector Similarity Search

Ember includes optional HNSW-backed vector similarity search, compiled in with `--features vector`. The index implementation is provided by the usearch library [10] (C++ via FFI), which ships SIMD-optimized distance computations.



(a) Query throughput (b) Memory usage

Fig. 5: Vector similarity search: 100K random vectors, 128 dimensions, cosine metric, $k{=}10$ kNN. HNSW parameters: $M{=}16$, ef_construction=64. Ember's 8-shard mode achieves 4.7× the query throughput of ChromaDB at 4–6× lower memory consumption.

TABLE III: Vector performance scaling by shard count. Insert throughput scales near-linearly with shards because each builds an independent HNSW index. Query latency improves with sharding—smaller per-shard indexes mean faster graph traversal.

Shards	Insert (vec/s)	Query (q/s)	p99 (ms)
1	2,432	1,217	0.99
4	3,742	1,175	0.97
8	5,482	1,793	0.62

Figure 5 compares query throughput and memory usage for 100K vectors at 128 dimensions. Ember in 8-shard mode achieves 1,793 queries/s with a p99 of 0.62 ms, consuming only 31 MB of memory. ChromaDB achieves 381 queries/s at 139 MB; pgvector achieves 782 queries/s at 178 MB; Qdrant achieves 560 queries/s at 168 MB.

The throughput advantage comes from Ember's thread-per-core architecture: when vectors are distributed across 8 shards (one per key), each shard builds and queries an independent HNSW index on its own core. Table III shows how insert throughput scales nearly linearly with shard count.

On the standard SIFT1M benchmark (1M vectors, 128 dimensions, 10K queries), Ember achieves a recall@10 of 0.9339 with $M{=}16$ and ef_construction=64—competitive with dedicated vector databases at the same HNSW parameters. Insert throughput of 3,755 vectors/s and query throughput of 1,596 queries/s confirm that the system handles million-scale indices.

F. Persistence and Encryption Overhead

Figure 6 shows the cost of durability. With `appendfsync everysec`, Ember's SET throughput drops to ∼70% of baseline (1,240K/s), closely matching Redis's proportional reduction (∼69%). The absolute numbers remain substantially higher: Ember at 1,240K/s versus Redis at 667K/s.

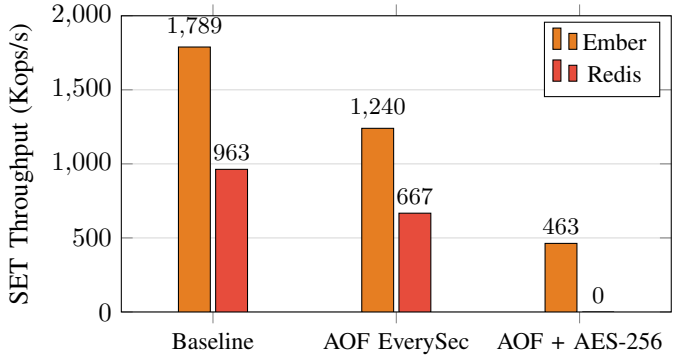


Fig. 6: Persistence and encryption overhead at P=16. Both systems show ∼30% throughput reduction with AOF enabled. AES-256-GCM encryption adds 60% overhead to SET writes at high pipeline depths; GET throughput is unaffected since reads are served from the in-memory keyspace. Redis does not offer native encryption at rest (value omitted).

TABLE IV: Per-command throughput across data types at P=1 (`redis-benchmark`, 50 clients). All types achieve consistent throughput, confirming that per-command overhead is dominated by channel routing rather than data structure operations.

Command	ops/s
SET	455,677
GET	501,924
LPUSH	465,777
SADD	462,665
ZADD	450,479
HSET	462,443
HGET	473,517

AES-256-GCM encryption at rest adds 60% overhead to SET writes at P=16, where sustained write volume stresses the encryption pipeline. At P=1, the encryption overhead is negligible because individual requests are not bottlenecked by AES throughput. Critically, GET throughput is completely unaffected by encryption because reads are served from the in-memory keyspace—the ciphertext is only on disk.

G. Data Type and Transaction Throughput

Table IV demonstrates throughput uniformity across data types. All commands achieve 450–500K ops/s at P=1, confirming that per-command overhead is dominated by the channel routing mechanism rather than by the underlying data structure operations. This is a desirable property: it means workloads can freely mix data types without encountering unexpected performance cliffs.

Table V quantifies transaction overhead. Wrapping a SET in MULTI/EXEC costs approximately 30% for both systems, indicating that transaction bookkeeping overhead is similar. Ember maintains a 1.5× throughput advantage in both the bare and transactional cases.

TABLE V: Transaction overhead: bare SET vs. MULTI-/SET/EXEC at P=1 (50 clients, 100K requests). Both systems incur ∼30% overhead from transaction wrapping.

Test	Ember	Redis
Bare SET	167,064	108,698
MULTI/SET/EXEC	111,195	76,798
Overhead	33%	29%

TABLE VI: Pub/sub throughput and fan-out scaling. 10K messages per test. Total fan-out delivery rate increases from 8.6K to 24.7K msg/s as subscribers grow from 1 to 100. PSUBSCRIBE (pattern matching) performs identically to SUBSCRIBE. Message size has minimal impact.

Test	Pub msg/s	Fan-out msg/s	p99 (ms)
1 sub, 64 B	8,639	8,639	0.23
10 sub, 64 B	1,751	17,511	3.39
100 sub, 64 B	396	24,731	29.58
1 sub, 1 KB	9,265	9,265	0.24
10 sub, 1 KB	1,712	17,117	3.50
100 sub, 1 KB	399	23,779	31.33

H. Pub/Sub Fan-Out

Table VI presents pub/sub throughput across subscriber counts and message sizes. The key observation is that fan-out delivery throughput scales well: total message delivery rate increases from 8.6K to 24.7K msg/s as subscribers grow from 1 to 100, even though per-publisher throughput drops proportionally (each PUBLISH now fans out to 100 receivers). Message size (64 B vs. 1 KB) has minimal impact, indicating that the bottleneck is connection management and delivery scheduling rather than serialization bandwidth.

Pattern-based subscriptions (`PSUBSCRIBE`) perform identically to exact-match subscriptions—the glob matching overhead is negligible relative to the network I/O cost of delivering messages to subscribers.

I. Protocol Comparison: RESP3 vs. gRPC

Table VII compares RESP3 and gRPC for standard operations. RESP3 pipelining is 7–9× faster than sequential RESP3 and 15–21× faster than gRPC unary calls. The overhead of gRPC unary RPCs comes from HTTP/2 framing, header compression, and per-request protobuf serialization.

However, throughput is not the sole consideration. gRPC provides type-safe APIs with generated client libraries, bidirectional streaming for vector queries (where it is 16% faster than RESP3), and native support for protobuf-encoded payloads. For applications that already use gRPC as their service mesh protocol, the ability to communicate with the cache in the same protocol eliminates a serialization boundary.

VI. Discussion

A. Where Shared-Nothing Wins

The throughput results in Section V are not incidental — they follow directly from architectural properties. Four factors compound:

TABLE VII: RESP3 vs. gRPC throughput for standard SET/GET operations (100K requests, 64 B values). RESP3 pipelining is the fastest mode; gRPC unary has higher per-request overhead from HTTP/2 framing but provides type-safe APIs and streaming RPCs.

Protocol / Mode	ops/s	p50 (ms)	p99 (ms)
RESP3 SET (sequential)	11,361	0.087	0.116
RESP3 GET (sequential)	12,244	0.081	0.108
RESP3 SET (pipelined)	75,726	0.013	0.015
RESP3 GET (pipelined)	106,422	0.009	0.011
gRPC SET (unary)	5,031	0.190	0.267
gRPC GET (unary)	5,119	0.187	0.262

1) **Lock-free hot path.** No mutex, spinlock, or atomic compare-and-swap appears on the data path. The only synchronization primitives are the channel send/receive operations, which are amortized across pipeline batches via burst draining.
2) **Cache-line isolation.** Each shard's hash map, memory tracker, and expiry state reside in shard-local memory. No other thread writes to these addresses, so the L1/L2 cache hierarchy is never invalidated by remote stores. This is the "mechanical sympathy" that shared-state architectures sacrifice.
3) **Natural pipeline parallelism.** The dispatch-collect pattern converts pipeline depth into shard-level parallelism automatically. A pipeline batch touching N shards achieves up to $N\times$ concurrency with no additional application logic.
4) **Per-shard persistence.** AOF and snapshot files are per-shard, enabling embarrassingly parallel recovery. A system with 8 shards recovers 8 AOF files simultaneously on 8 cores.

B. Where It Trades Off

Intellectual honesty requires acknowledging three areas where Ember's architecture is not strictly superior:

Single-request latency. At $P{=}1$, Ember's p99 latency is approximately $2\times$ that of Redis (1.247 ms vs. 0.647 ms for SET). The $\sim$0.6 ms delta is the inherent cost of channel traversal: mpsc send, shard wakeup, command execution, oneshot reply. Redis avoids this by executing commands directly in the event loop thread. For latency-critical, unpipelined workloads, Redis remains the better choice.

Memory overhead. Ember uses 4–26% more memory per key than Redis, depending on data type. The overhead comes from per-entry metadata fields needed for LRU tracking and memory accounting. Redis achieves lower overhead through compact encodings (ziplist, listpack) that pack small collections into contiguous byte arrays. Implementing these encodings in Ember is a potential future optimization.

Feature breadth. Ember supports 190+ commands versus Redis's 200+. Lua scripting, streams, and multiple databases are intentionally excluded. These are conscious scope decisions—not bugs—but they limit Ember's applicability for workloads that depend on server-side scripting or stream processing.

C. When to Choose Ember

The evaluation data suggests three workload profiles where Ember's architecture provides a compelling advantage:

1) **Throughput-dominated pipelines.** Batch ingestion, cache warming, ETL cache layers, and any workload where clients can pipeline commands. The dispatch-collect pattern yields 1.8–2.2$\times$ throughput gains that compound across millions of operations per second.
2) **Vector-augmented caching.** Applications that combine traditional key-value caching with similarity search—recommendation engines, RAG pipelines, semantic caches—benefit from co-locating both workloads in a single system rather than operating a cache plus a separate vector database. The 4–6$\times$ memory advantage is particularly significant when vector indices must fit in memory.
3) **Multi-core utilization.** Environments where CPU cores are provisioned but underutilized by single-threaded caches. Ember's 9$\times$ scaling on 8 cores means fewer instances are needed to serve the same aggregate throughput, reducing operational complexity and cost.

Conversely, workloads that require sub-millisecond p99 latency at $P{=}1$, Lua scripting, or stream processing should continue to use Redis.

D. The Role of Rust's Ownership Model

It is worth discussing whether the shared-nothing architecture *requires* Rust, or whether it is merely convenient. In principle, any language can implement shard isolation through disciplined engineering. In practice, shared-nothing invariants are notoriously fragile in languages with unrestricted aliasing.

A C++ implementation of the same architecture would need to rely on coding conventions—"never pass a pointer to another thread's keyspace"—enforced by code review. A single violation would introduce a data race invisible to the compiler. In Rust, this is a compile-time error: the borrow checker rejects any code path that would allow two threads to hold mutable references to the same keyspace partition.

This is not a minor convenience. It means the shared-nothing property is maintained automatically as the codebase grows, as new contributors join, and as the system evolves. The compiler does the enforcement work that would otherwise require constant human vigilance.

E. Threats to Validity

Our evaluation was conducted on a single hardware configuration (GCP c2-standard-8). Results may differ on ARM architectures, NUMA topologies, or machines with different core counts. All benchmarks were run on loopback, eliminating network latency; real deployments involve network hops that would compress the relative advantage. Redis and Dragonfly are mature, production-hardened systems; Ember's performance

numbers should not be interpreted as a claim of operational parity.

Additionally, both Redis and Dragonfly offer features that Ember does not: Lua scripting, streams, modules, and in Dragonfly's case, sophisticated memory management via DashTable that achieves $\sim 25\%$ of Redis's memory usage for equivalent workloads. Ember's performance advantages must be weighed against these functional differences.

VII. Related Work

Redis [1] is the most widely deployed in-memory cache, with an ecosystem spanning hundreds of client libraries and a decade of production deployment at scale. Its single-threaded event loop provides an elegant programming model: commands execute atomically without any concurrency control, and the codebase ($\sim$160K lines of C) is remarkably readable for a system of its maturity. Redis 6.0 introduced I/O threads that offload network reads and writes to a configurable thread pool, but command execution remains serial on the main thread. Redis Cluster distributes keys across nodes using 16,384 hash slots with gossip-based failure detection and manual resharding.

Dragonfly [2] is a modern multi-threaded Redis alternative written in C++. It uses a novel DashTable data structure with fine-grained, per-bucket locking and achieves fork-free snapshotting—a significant advantage for large datasets where Redis's COW-based fork can cause latency spikes and double memory usage. Dragonfly supports the full Redis API (200+ commands) and achieves $\sim 25\%$ of Redis's memory usage through its compact hash table design. Our benchmarks show that despite these sophisticated optimizations, the shared-state architecture limits Dragonfly's throughput relative to Ember's shared-nothing approach.

Memcached [4] pioneered distributed in-memory caching with a multi-threaded architecture and slab allocator. It supports only string values and lacks persistence, transactions, and rich data structures. Its simplicity makes it attractive for pure-caching workloads, but the absence of data types limits its applicability.

ScyllaDB [5] and the Seastar framework [6] established the thread-per-core, shared-nothing paradigm in the database world. ScyllaDB applies this architecture to a Cassandra-compatible wide-column store, achieving throughput improvements of up to $10\times$ over Apache Cassandra. Ember draws direct inspiration from this approach: the principle that each core should own its data exclusively, communicating only through message passing, translates naturally from persistent storage to volatile caching.

Vector databases. The proliferation of embedding-based applications has produced a generation of purpose-built vector databases. Qdrant [14] is a Rust-based vector search engine with rich filtering and payload storage. ChromaDB [15] targets AI/ML workflows with an embedding-first API designed for rapid prototyping. pgvector [16] extends PostgreSQL with HNSW and IVFFlat indices, offering the advantage of SQL-based querying alongside vector search. These systems provide richer query APIs—filtered search, hybrid queries, metadata indexing—at the cost of higher per-vector memory overhead (4–6$\times$ more than Ember for equivalent workloads). Ember positions vector search as an *optional extension* to a general-purpose cache, which is appropriate for applications that combine similarity search with traditional key-value operations.

The HNSW algorithm [9] provides the approximate nearest neighbor index structure used by Ember and most of the systems above. Its hierarchical graph structure achieves logarithmic search complexity with tunable accuracy/speed trade-offs via the M and ef parameters. The SWIM protocol [7] provides Ember's failure detection mechanism, offering $O(n)$ total message load compared to $O(n^2)$ for traditional heartbeat-based protocols. Raft [8] provides Ember's consensus layer for topology management; its understandability relative to Paxos was a factor in its selection.

VIII. Conclusion

We have presented Ember, a distributed cache that applies the shared-nothing, thread-per-core paradigm to in-memory key-value storage. By assigning each CPU core an exclusive keyspace partition and routing commands through bounded channels, Ember achieves $1.9\times$ SET and $1.8\times$ GET throughput over Redis, and $2.2\times$ over Dragonfly, on an 8-vCPU instance. For vector similarity workloads, the architecture enables $4.7\times$ query throughput at 4–6$\times$ lower memory than ChromaDB.

These results confirm a hypothesis that, in retrospect, should not be surprising: a cache that does no locking on the data path will outperform one that does, provided the routing overhead is amortized through pipelining. The more interesting finding is the degree to which Rust's ownership model makes the shared-nothing constraint enforceable rather than merely aspirational. The compiler, rather than code review, ensures that no shard touches another's data.

Ember is open source under the Apache 2.0 license. The codebase comprises approximately $25{,}000$ lines of Rust (excluding tests and comments) and supports 190+ commands across strings, lists, sorted sets, hashes, sets, bitmaps, transactions, pub/sub, clustering, persistence, TLS, ACL, and optional vector similarity search and protobuf storage.

Future work. We plan to add geographic data types (GEO commands), investigate WASM-based server-side extensions as an alternative to Lua scripting, evaluate skip list implementations for O(log n) rank queries on sorted sets, and expand the evaluation to cover NUMA topologies and ARM architectures.

References

[1] S. Sanfilippo, "Redis: An in-memory data structure store," 2009–2024. [Online]. Available: https://redis.io

[2] R. Gershman *et al.*, "Dragonfly: A modern in-memory datastore," DragonflyDB, 2022. [Online]. Available: https://dragonflydb.io

[3] S. Sanfilippo, "Redis Cluster specification," 2014. [Online]. Available: https://redis.io/docs/reference/cluster-spec/

[4] B. Fitzpatrick, "Distributed caching with memcached," *Linux Journal*, vol. 2004, no. 124, p. 5, 2004.

[5] A. Rappaport *et al.*, "ScyllaDB: A Cassandra-compatible NoSQL database with shared-nothing architecture," ScyllaDB Inc., 2015. [Online]. Available: https://scylladb.com

[6] A. Rappaport *et al.*, "Seastar: High-performance server-side application framework," 2014. [Online]. Available: https://seastar.io

[7] A. Das, I. Gupta, and A. Motivala, "SWIM: Scalable weakly-consistent infection-style process group membership protocol," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2002, pp. 303–312.

[8] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (ATC)*, 2014, pp. 305–319.

[9] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[10] A. Vardanyan, "USearch: Smaller & faster single-file vector search engine," Unum Cloud, 2023. [Online]. Available: https://github.com/unum-cloud/usearch

[11] G. Fowler, L. C. Noll, K. Vo, and D. Eastlake, "The FNV non-cryptographic hash algorithm," IETF Internet-Draft, 2019.

[12] J. Evans, "A scalable concurrent malloc(3) implementation for FreeBSD," in *Proc. BSDCan*, 2006.

[13] S. Sanfilippo, "Random notes on improving the Redis LRU algorithm," 2016. [Online]. Available: http://antirez.com/news/109

[14] Qdrant Team, "Qdrant: Vector similarity search engine," 2021. [Online]. Available: https://qdrant.tech

[15] Chroma Team, "Chroma: The AI-native open-source embedding database," 2023. [Online]. Available: https://www.trychroma.com

[16] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2021. [Online]. Available: https://github.com/pgvector/pgvector

[17] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[18] Tokio Contributors, "Tokio: An asynchronous runtime for Rust," 2016–2024. [Online]. Available: https://tokio.rs

[19] The Rust Team, "The Rust programming language," Mozilla Research, 2015–2024. [Online]. Available: https://www.rust-lang.org